

Reflection Report

Course: SE3004 Software Testing

Student: Muhammed Maruf Ulaş

Date: May 14th, 2026

1. Introduction

This project was about testing a Java e-commerce library called ShopEasy. We had to apply five different testing techniques from the textbook on the same codebase:

- specification-based testing
- structural testing
- design by contract
- property-based testing
- mock-based testing

At first, I thought testing was mostly just writing a bunch of `assertEquals` calls. Going through all five techniques showed me that different approaches catch completely different kinds of bugs.

2. Task Reflections

2.1 Task 1 — Specification-Based Testing (PriceCalculator)

For this task, I had to test:

`PriceCalculator.calculate(basePrice, discountRate, taxRate)`
using domain testing.

I created equivalence partitions for the three inputs:

- `basePrice`: zero, positive, large values
- `discountRate`: 0%, typical values, 100%
- `taxRate`: 0%, typical values, 100%

Then I selected boundary values. For percentages, the obvious boundaries are 0 and 100, but I also tested off-points like 0.01 and 99.99.

I used `@ParameterizedTest` with `@CsvSource` for the typical cases since the assertion structure was identical across most tests.

The thing that caught me off guard was floating-point precision.

I originally wrote:

```
assertThat(result).isEqualTo(2.40);
```

for a test involving bananas priced at 0.80 with quantity 3.

The test failed because Java calculated:

`0.80 * 3 = 2.4000000000000004`
instead of exactly 2.40.

Because of that, I switched to:

`isCloseTo(expected, within(0.001))`
for most assertions.

I honestly did not expect floating-point precision to matter in such a simple calculator, but it did.

2.2 Task 2 — Structural Testing (ShoppingCart)

I started by writing tests based on the `ShoppingCart` Javadoc — mostly happy-path tests for:

- adding items
- removing items
- applying discounts
- calculating totals

Then I ran:

`mvn jacoco:report`
and checked the JaCoCo report.

My initial branch coverage was only around **60%**, which was disappointing because I thought the tests were already pretty thorough.

The uncovered branches were:

- `addItem` when the product is not already in the cart
- `removeItem` on an empty cart
- `total()` with multiple products in the cart

To cover those branches, I added targeted tests like:

`addItem_loopProductNotFound_addsNewItem`
and:

`total_multipleItems_loopExecutesMultipleTimes`

One thing this task made very obvious is that you cannot reliably guess which branches are uncovered. The JaCoCo HTML report makes it extremely clear because the missed lines stay highlighted in red.

After adding the missing tests, the final branch coverage reached:

- **82%**

2.3 Task 3 — Design by Contract (ShoppingCart & PriceCalculator)

This task was confusing at first because we had to use `assert` statements instead of throwing exceptions like `IllegalArgumentException`.

I was not sure why we would use assertions since they only work when Java runs with:

```
-ea
```

enabled and get ignored in production.

After reviewing the lecture material again, I understood that contracts are meant to act as debugging tools rather than runtime validation.

The straightforward contracts were the preconditions:

```
assert product != null : "product must not be null";
assert basePrice >= 0 : "basePrice must be >= 0";
```

The annoying contract was the `applyDiscount` post-condition.

The requirement says the discounted result should be less than `total()` whenever `discountRate > 0`.

But if the cart is empty:

```
rawTotal == 0
discounted == 0
```

so the condition fails because both values are equal.

I spent a while trying to figure out how to write the assertion without breaking empty-cart scenarios.

Eventually I used:

```
assert discountRate == 0 || rawTotal == 0 || discounted <
rawTotal;
```

Writing `ContractTest.java` was actually satisfying because every method needed two perspectives:

1. valid input should succeed
2. invalid input should throw `AssertionError`

For invalid-input cases, I used:

```
assertThatThrownBy(...).assertInstanceOf(AssertionError.class)
```

It made me realize I normally spend much more time testing what should happen than what should never happen.

2.4 Task 4 — Property-Based Testing (jqwik)

This was probably the weirdest task because I had never used property-based testing before.

Instead of testing specific examples like:

```
calculate(100, 10, 20)
```

you define properties that should hold for all valid inputs.

I defined six properties:

- Final price is never negative
- Higher discount never increases price
- 0% discount and 0% tax returns the base price
- Adding product A then B gives the same total as B then A
- Adding then removing a product returns an empty cart
- Cart total never becomes negative

jqwik generated 1000 random test cases for each property.

None of them failed, which was reassuring, although it also made me wonder whether some properties were too weak.

I had to create a custom:

```
@Provide
```

method called:

```
validProducts()
```

because generating completely random values would constantly violate the `Product` constructor requirements.

I used:

```
Arbitraries.strings().alpha()
```

for IDs and:

```
Arbitraries.doubles().between(0.01, 500.0)
```

for prices.

The commutativity property was the most interesting to me because it checks whether the total stays consistent regardless of insertion order.

That is not something I would normally think about when writing example-based unit tests.

2.5 Task 5 — Mocks & Stubs (OrderProcessor)

Before writing tests, I sketched the dependency structure:

```
OrderProcessor
├── InventoryService
└── PaymentGateway
```

I mocked both dependencies with Mockito so I could test `OrderProcessor` without setting up a real database or payment API.

The most important scenario was inventory failure.

If any item is unavailable:

1. the method should return `null`
2. `charge()` should never be called

I verified this using:

```
verify(paymentGateway, never()).charge(anyString(),
anyDouble());
```

Without mocks, this test would be extremely difficult because it would require:

- a real inventory system
- a real payment service
- a way to intercept payment calls

With Mockito, the same scenario became a very small isolated unit test.

At the same time, I understand why mocking is not always enough.

If the real `PaymentGateway` changes its API or behaves differently over actual network requests, mock tests will not detect those problems.

Mocking is excellent for isolated logic testing, but it does not replace integration testing.

3. Overall Lessons

The biggest thing I learned is that one testing technique is never enough.

My specification tests covered obvious boundaries, but they would not necessarily catch strange cases like:

```
discountRate = 33.333
```

Property-based testing is much better suited for discovering those kinds of edge cases.

Structural testing forced me to examine branches I ignored completely at first, and mocking allowed me to test scenarios that would otherwise require a lot of infrastructure.